Engineering Applications of Artificial Intelligence 167 (2026) 113780

Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai

Research paper

# Multi-agent reinforcement curriculum learning for real unmanned ground vehicles

Timothy Mead [a], Zhe Wang [b], Ernest Foo [b], Jin Song Dong [c], Naipeng Dong [d], Ryan Ko [d], Abigail M.Y. Koay [e], Kien Nguyen [f], Yue Xu [f], Junae Kim [g], Stephen Bornstein [a], Zhe Hou [b,*]

[a] Cyborg Dynamics Engineering, Brisbane, 4172, QLD, Australia
[b] Griffith University, Brisbane, 4111, QLD, Australia
[c] National University of Singapore, 119077, Singapore
[d] The University of Queensland, Brisbane, 4072, QLD, Australia
[e] University of the Sunshine Coast, 4556, QLD, Australia
[f] Queensland University of Technology, Brisbane, 4000, QLD, Australia
[g] Defence Science and Technology Group, Brisbane, 4072, QLD, Australia

ARTICLE INFO

*Keywords:*
Multi-agent
Swarm
Reinforcement learning
Curriculum learning
Federated learning
Unmanned ground vehicles
Autonomous systems
Robotics

ABSTRACT

This paper investigates the use of deep reinforcement learning (DRL) for the control of mobile robot teams within the context of navigation and task-based collaborative scenarios. We apply a DRL policy with a tailored neural network architecture as a solution to control, path planning, and higher-level guidance tasks. Our network architecture was trained using a unique multi-stage curriculum that progresses from single-agent navigation, to multi-agent pathfinding with obstacles, and finally to a complex collaborative firefighting scenario. This structured approach accelerates training convergence by systematically building sophisticated collaborative behaviours upon foundational skills, which enhances training stability and guides the agents towards learning effective and coordinated strategies The policy evaluation was conducted in both simulation and hybrid simulation-physical demonstrations utilising a real unmanned ground vehicle (UGV). The policy presented is capable of achieving multi-agent navigation tasks with a 95.83% accuracy in our testing environments, and has demonstrated emergent multi-agent behaviours. In more complex collaborative firefighting scenarios, the policy also demonstrated superior performance than baselines in reaching goals, e.g., navigating and extinguishing two fires with a 99.67% success rate, suggesting its strong potential for real-world deployment.

## 1. Introduction

The inherent dangers and dynamic complexities of emergency scenarios present formidable challenges to human responders and operators. Robotic systems are a transformative technology for enhancing the capabilities of human teams, particularly in scenarios that are too hazardous or physically demanding. By undertaking dangerous tasks, these systems not only address the escalating scale of certain disasters where human capacity is limited, but also alleviate the psychological burden on responders. This evolution from remotely operated tools to increasingly autonomous systems necessitates sophisticated AI, moving beyond simple programmed behaviours to systems capable of complex reasoning and adaptive action. Therefore, autonomous decision-making and mission control for unmanned vehicles is a significant problem (Miller et al., 2024).

Within the application of mobile robotics, the task of point-to-point navigation to a desired location is often a prime focus (Jestel et al., 2021). Unmanned Ground Vehicles (UGVs) have garnered significant attention as robust platforms for ground missions, capable of traversing challenging terrains to perform reconnaissance, surveillance, identify and mitigate hazards, and transport essential payloads in environments otherwise inaccessible or too dangerous for human teams. While individual UGVs offer substantial benefits, the concept of deploying multiple UGVs presents a paradigm shift in robotic systems. Multi-agent robotics requires decentralised control, enhanced scalability, operational flexibility, and system robustness. This shift towards multi-agent systems reflects a broader strategy favouring collective intelligence and resilience over reliance on single, highly complex, and potentially vulnerable assets.

* Corresponding author.
*E-mail address:* z.hou@griffith.edu.au (Z. Hou).

https://doi.org/10.1016/j.engappai.2026.113780
Received 21 July 2025; Received in revised form 22 October 2025; Accepted 6 January 2026
Available online 8 January 2026
0952-1976/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

**Table 1**
Progression of the Multi-stage Training Curriculum.

| Stage | Objective | Key Rewards |
|---|---|---|
| **One:** Single-Agent Movement | Navigate to a single target point. | Making progress towards and reaching the target. |
| **Two:** Multi-Agent Pathfinding | Navigate to targets while avoiding obstacles and other agents. | Reaching the goal. (Penalties for collisions and proximity). |
| **Three:** Collaborative Firefighting | Coordinate as a team to extinguish multiple fires, some requiring multiple agents. | Hosing and extinguishing fires. (Penalties for collisions). |

Addressing these complexities requires advanced AI solutions. Deep Reinforcement Learning (DRL) has proven to be a powerful approach (Baker et al., 2020), enabling agents to learn optimal decision-making strategies through direct interaction with their environment, making it highly suitable for autonomous navigation and control in complex, unpredictable settings. For multi-agent systems, we adopt Multi-Agent Reinforcement Learning (MARL), which specifically tackles scenarios where multiple agents learn concurrently in a shared environment (Yu et al., 2023). MARL is particularly well-suited for decentralised coordination, as it allows agents to develop adaptive strategies based on local observations and interactions. However, MARL introduces its own challenges, including non-stationarity, where the environment changes as other agents learn, the credit assignment problem, scalability issues, partial observability, and the overarching complexity of ensuring coherent and effective coordination rather than detrimental interference.

Training MARL agents for such complex tasks is impractical on physical hardware due to safety risks and high operational costs, necessitating the use of simulation. This reliance on simulation, however, introduces the notorious "simulation-to-reality" (sim-to-real) gap, where policies fail on physical hardware due to unmodelled dynamics (Wang and Zhao, 2025). To address both the learning complexity and the sim-to-real challenge simultaneously, our work centres on a carefully designed Curriculum Learning (CL) strategy (Xiao et al., 2021) and a physics simulation tuned to empirically collected data on the UGVs used in this work. An effective curriculum must not only sequence tasks by difficulty but also strategically incorporate elements to bridge this reality gap. As detailed in Table 1, our curriculum progresses logically from foundational single-agent navigation, to coordinated multi-agent pathfinding with obstacle avoidance, and finally culminates in a complex, collaborative firefighting scenario requiring specialised actions and emergent task allocation. This structured progression allows policies to build upon previously learned skills, accelerating convergence on sophisticated behaviours. The successful deployment of these learned policies on physical hardware, validated via Hardware-in-the-Loop (HIL) testing, is therefore the critical test of our integrated approach. The novelty of this work lies in proposing and validating this integrated framework solution that spans MARL, CL, and sim-to-real transfer for a demanding real-world application.

As a comparison to a state-of-the-art method, we have also implemented federated learning for multi-agent decision-making, and trained it through the same curriculum. However, as shown in experiments, federated learning yielded inferior results, so we do not formally incorporate it as a part of our proposed method.

The main contributions of this paper are as follows:

- A new DRL architecture for individual agents that integrates high-level strategic navigation with low-level reactive path-planning and control, enabling them to manage complex mission demands.
- A multi-agent learning approach that combines a shared network with curriculum learning to stably guide agents through progressively difficult cooperative tasks, helping learn sophisticated coordinated behaviours.
- The practical transfer of a DRL policy to a real UGV, validated with a hybrid, hardware-in-the-loop testing architecture, demonstrating promising results for real-world application and a methodology for addressing the sim-to-real gap.

The remainder of this paper is organised as follows: Section 2 reviews related work, Section 3 describes our learning approach, Section 4 analyses experimental results, and Section 5 concludes the paper.

## 2. Related work

In this section we provide an overview of current studies on DRL related to mobile robot tasks, different training approaches, and work in multi-agent scenarios. Throughout, we critically compare these existing methods to our proposed framework to highlight our key contributions.

*Control paradigms for multi-agent systems.* The coordination of multiple robots is addressed through various paradigms. Traditional, often model-based, methods offer robust performance guarantees when system dynamics are well-defined. For instance, some studies develop resilient, distributed adaptive controllers for physically complex systems like helicopters with slung-loads, using neural networks to approximate unknown nonlinearities within the control law to ensure stability (Peng et al., 2025). Other work presents decentralised Iterative Learning Control (ILC) algorithms that use recursive, Broyden-like update laws to efficiently optimise control inputs for repetitive collaborative tracking tasks (Gao et al., 2025). However, these approaches can be brittle in highly dynamic, non-repetitive environments. For the emergency response scenarios central to our work, we therefore selected DRL for its adaptability in learning complex behaviours directly from environmental interaction.

*Deep reinforcement learning (DRL).* DRL has guided UGV decision-making in applications like self-driving cars, robotic manipulation, and uncertain path planning. This has led to advanced emergent behaviours, such as collaborative tool use (Baker et al., 2020), and has even surpassed human performance in drone racing (Kaufmann et al., 2023).

However, single-agent DRL treats other robots as mere environmental elements, hindering performance in collaborative tasks and prompting extensions to multi-agent scenarios. To address this, researchers are adapting DRL for multi-UGV teams. One approach is centralised training with decentralised execution (CTDE), where simulated teams train with shared state information to develop decentralised policies, leading to emergent group behaviours like flocking or target tracking (Li et al., 2024). Another method uses transfer and meta-learning, allowing a policy learned by one robot to be adapted by others, which accelerates learning in new environments (Orr and Dutta, 2023).

*Multi-agent reinforcement learning (MARL).* MARL adapts RL for multiple interacting agents to learn coordinated strategies. Standard RL algorithms struggle in multi-agent settings due to non-stationarity. To counter this, many MARL methods use Centralised Training with Decentralised Execution (CTDE), such as MADDPG (Li et al., 2019) and MAPPO (Alagha et al., 2022), enabling agents to optimise shared objectives like area coverage while acting independently.

Adapting MARL to real-world conditions has shown significant benefits. For instance, Yu et al. (2023) used an asynchronous multi-agent PPO that allowed exploration robots to act and learn at different rates, substantially improving exploration efficiency in physical deployments.

MARL is also applied to complex coordination problems like task allocation. For example, the work in Su et al. (2024) uses a multi-agent Actor-Critic (AC) framework with a parameter-sharing mechanism to generate cooperative siege tactics for UCAVs. In a multi-object transport scenario with unknown weights, Shida et al. (2025) developed

a framework where robots share their "task experience". This allows the team to learn from each other's attempts and avoid deadlocks on objects too heavy for a single agent, showing scalability and adaptability to new robots. While these methods demonstrate effective coordination, they often rely on specialised communication protocols or task-specific architectures. Our primary MARL contribution is a novel neural network architecture for the agent's policy itself, which integrates perception, self-state, and the states of other agents and targets into a unified model that outputs both low-level control and high-level navigational decisions without a separate planner.

Through learned joint policies, MARL can produce emergent cooperative behaviours – such as turn-taking or escorting – that were not explicitly programmed (Phan, 2023). While promising, these emergent behaviours introduce challenges in predictability and safety.

*Federated reinforcement learning (FRL).* In FRL, agents train local policies, periodically send their model parameters to a central server for aggregation into a global policy, and then download this updated policy to continue local training. This cycle facilitates team-wide knowledge sharing, as agents benefit from others' experiences without direct data access (Upadhyay et al., 2024).

The architecture design is a critical consideration. Many FRL systems use a central server for synchronous updates, as seen in the federated deep Q-learning (FLDQN) system for traffic-aware routing, where vehicles cooperatively optimised routes and converged faster than single-agent methods without sharing raw traffic data (Mamond et al., 2025). Alternative architectures include asynchronous FL, which reduces robot idle time by allowing updates without waiting for others (Upadhyay et al., 2024), and hierarchical FL, which uses intermediate aggregators to cut latency in large fleets.

FRL has proven effective even under severe communication constraints. A study on robotic swarms found that a federated approach to training a Deep Deterministic Policy Gradient (DDPG) algorithm (Lillicrap et al., 2019) improved training stability over other shared network methods (Na et al., 2022).

*Curriculum learning for multi-agent coordination.* Curriculum learning (CL) enhances convergence in complex MARL systems by structuring training in stages, from simple to difficult tasks. For example, Xiao et al. (2021) employed a curriculum that incrementally increased the required precision to obtain a reward as the agent's policy improved.

For long-horizon tasks with sparse rewards, CL is particularly effective at generating intermediate goals. In a challenging collaborative task where a UAV had to land on a moving UGV, Wang et al. (2023) used CL to progressively increase difficulty. The system automatically adjusted the UGV's speed, added turns, and intensified wind disturbances as the UAV's landing policy improved, with a learned discriminator determining when to advance the curriculum.

However, designing an effective curriculum is non-trivial, as a poorly structured sequence can confuse agents and degrade performance (Wang et al., 2023). Our contribution is not merely the use of CL, but the design of a specific three-stage curriculum structured to bootstrap complex, collaborative behaviours (i.e., emergent task allocation in firefighting) from fundamental individual skills (i.e., navigation and obstacle avoidance) within a single, shared policy.

*Navigation tasks.* For local point-to-point navigation, DRL has been used to develop decentralised policies for multi-robot systems. Jestel et al. (2021) demonstrated that even non-recurrent DRL networks can produce emergent collaborative behaviours like yielding and zipper merging in tasks such as navigating intersections. While their approach performed well in known environments, its success rate declined in unseen settings. Our method directly addresses this by incorporating randomised obstacles in training to foster a more robust and generalisable policy. To handle a variable number of obstacles during collision avoidance, Everett et al. (2021) introduced a DRL architecture using Long Short-Term Memory (LSTM) layers, a technique also shown to be effective for point-to-point navigation by Wen et al. (2021) and Alonso et al. (2020). The navigation task has also been extended to include real-time traversability assessment in rough terrain Zhang et al. (2018).

More recently, Wei et al. (2023) presented a DRL-based mapless navigation method using LiDAR and odometry data to output steering commands. By incorporating a heuristic correction mechanism to stabilise learning, their policy enabled a UGV to handle dynamic obstacles like pedestrians and significantly outperformed classic planning baselines.

*Exploration and search & rescue.* In exploration tasks, such as searching a disaster site, agent teams must coordinate to maximise coverage while occasionally rendezvousing to share information under communication constraints (Tan et al., 2025). While classic heuristic methods are effective, they lack the adaptability of MARL, which can learn dynamic strategies. For example, the ACE system from Yu et al. (2023) addressed real-time multi-robot exploration by training a policy that accounts for asynchronous actions and robot dropouts. In contrast to such single-mission policies, our framework demonstrates the ability to handle heterogeneous objectives by integrating both navigation and collaborative intervention (firefighting) within a single mission. Despite these applications, practical MARL or FRL deployments in disaster scenarios are rare.

*Sim-to-real transfer and validation.* Transferring policies from simulation to physical hardware is a well-known challenge in robotics (Wang and Zhao, 2025). While many MARL studies remain purely in simulation, our work addresses this gap directly. Our contribution in this area is not only the successful deployment on a physical UGV, but also our methodology for validation. We propose and demonstrate a hybrid physical-cyber testing architecture, where a single physical UGV interacts with a swarm of simulated agents. This provides a practical and cost-effective method for rigorously evaluating complex swarm behaviours and validating sim-to-real transfer even with limited physical hardware.

*Research gaps.* Despite substantial progress of this field in recent years, several gaps in the literature remain:

- Current approaches typically train for fixed tasks like navigation (Jestel et al., 2021) or exploration (Yu et al., 2023). In contrast, real-world scenarios involve dynamic tasks, applications and team compositions. Our work addresses this by training a policy for a multi-objective mission that requires both navigation and task-specific collaboration.
- Many current DRL implementations focus on either low-level control or high-level planning. Our work contributes a unified DRL policy architecture that integrates both, addressing a key challenge in creating truly autonomous multi-agent systems.
- Most advanced MARL and FRL systems are validated solely in simulation. However, transitioning from simulation to real-world deployment is a critical step towards using these methods to solve real-world problems. More real-world testing is essential to address practical challenges that simulations cannot fully replicate. Our work addresses this practical challenge by deploying our trained algorithms on one of the most advanced UGV systems (BIA5) on the market.

## 3. Method

This study begins with a modification of existing neural network architectures that is applied to a single agent policy. This policy is then extended to a multi-agent scenario utilising shared network and federated learning approaches. We adopt curriculum learning in three stages using a uniquely developed reward structure at each stage to tune the model to our scenarios of increasing complexity, requiring emergent behaviours and collaborative task allocation for success. We utilise a simulator with suspension modelling tuned on empirically collected UGV data to facilitate an appropriate sim-to-real transfer of the network policy.

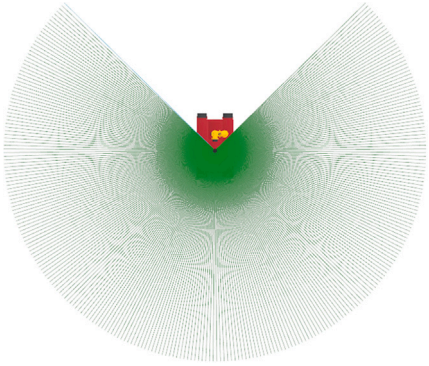


**Fig. 1.** Agent with visible LiDAR beams.

### 3.1. RL training algorithm

To develop a DRL policy for this task, Proximal Policy Optimisation (PPO) was chosen as the training algorithm due to its previously demonstrated performance benefits over other algorithms, such as TRPO or A2C (Schulman et al., 2017). The PPO training cycle iteratively collects experience by rolling out the policy, estimates the advantage of each action, and uses this to update the policy until performance converges. The use of PPO as a training algorithm specifically emphasises on improving training stability by clipping the size of policy updates during each optimisation cycle to a given bound, as large model updates are more likely to cause performance collapse. Our implementation of the PPO training algorithm was branched from the open-source CleanRL implementation (Huang et al., 2022) with modifications implemented to support various multi-agent learning architectures, multi-agent scenarios, and simulator integration, as detailed below.

### 3.2. Agent design

*Observation space.* The observation vector $o$ for each agent consists of four components,

$$o = (o_l, o_s, o_t, o_a). \tag{1}$$

This observation vector is inspired by the prior work done involving DRL multi-agent point-to-point navigation by Jestel et al. (2021), extended by passing the states of objects of interest, similar to the work done by Baker et al. (2020). Each observation vector is stacked with the previous three observation vectors before being passed into the RL model. The components of the observation vector are explained below.

Each UGV has a 2D LiDAR module that provides scans $o_l$ used for the detection of obstructions and boundaries of other agents. The LiDAR covers a 270° range with beams 1° apart, as shown in Fig. 1.

The self-state observation $o_s = (v, \omega, \imath)$ provides the agent with a measurement of its current linear $v$ and angular $\omega$ velocities and a Boolean signal indicating whether the agent is currently interacting with an objective $\imath$.

The positional states of a number $N_t$ of targets of interest are provided as $o_t$. These are represented as a relative normalised direction vector $(x, y)$ from the agent and the current distance $d$ between the agent and the target. The lack of a target is indicated with zeros $(0, 0, 0)$.

The state of a number $N_a$ of agents are provided as $o_a$. These states are comprised of the relative normalised direction $(x, y)$ towards the other agent, the current distance $d$ between the two agents, and a Boolean signal indicating whether the agent is interacting with an objective $\imath$. The intention of this observation was not to significantly improve obstacle avoidance (as the $o_l$ has been shown to provide sufficient information for this (Jestel et al., 2021; Surmann et al., 2020), but to instead allow the agents to consider the current formation of the team in its higher level decision making. The lack of an agent is indicated with zeros $(0, 0, 0, 0)$. The addition of this nearby agent state data increases the communication overhead between the agents, but limiting the policy to only require the $N_a$ nearest agents improves scalability for larger swarm sizes.

*Action space.* At each timestep the agent makes two continuous actions, consisting of a linear $v$ and angular $\omega$ velocity.

*Neural network architecture.* The neural network architecture for an RL agent is shown in Fig. 2. The navigation portion of the network follows a similar architecture to Jestel et al. (2021). This work expands upon this architecture by including additional observations in $o_s$, $o_t$, and $o_a$ to include data regarding current self activity, multiple mission-relevant targets, and multiple nearby agents. These additional observations each connect to a hidden layer before being concatenated together and passed through a second hidden layer.

As in Jestel et al. (2021) and Mnih et al. (2015), the last four observation vectors $o$ are stacked together as input to provide additional temporal information to the stateless network. The LiDAR observation $o_l$ passes through two 1D convolution layers with 16 and 32 filters, respectively, before passing to a fully connected layer of 256 units. The velocity $o_v$, target $o_t$, and agent $o_a$ observations each pass through a single fully connected layer of 32, 192, and 144 units, respectively. Each branch is then concatenated and passed to a final fully connected layer of 384 units. Each layer is preceded by a ReLU activation. The output of the network consists of the action $a$ and value $c$. These layer parameters were chosen due to their previously shown effectiveness in similar tasks (Jestel et al., 2021) and to provide a baseline comparison against our extension. For all training and evaluation, the RL policies were run at a sample period of $T = 0.5$s, which was shown to be suitable for the operation of our target UGV.

### 3.3. Multi-agent learning architectures

In extending the PPO training algorithm to a multi-agent scenario, the training data collected from the agents can be shared to improve the training speed and generalisation of an RL policy. This work has evaluated two separate methods of organising this data, shown in Fig. 3.

The Shared-Network approach (Na et al., 2022) utilises a single neural-network policy that is used by each agent on rollout. At the end of the rollout, the agent experiences that were collected in parallel by the different agents are concatenated into a single vector for policy optimisation. This approach requires the transmission of observation data from agents to a central server for optimisation and the redistribution of the updated policies for following training cycle. This is a suitable approach for offline training of policies, however, if online updates are required this may be unsuitable if the sharing of sensor data has security implications.

The Federated-Network approach (Na et al., 2022) provides a solution to the security issues of the Shared-Network approach in online scenarios. In this approach, each agent performs the rollout and optimisation PPO cycles individually, with each agent training its own unique neural-network policy. After every $T_f$ optimisation cycles, the policy weights are transmitted to a central server that first averages the weights $\theta_i$ from each of the $N$ policies to produce the average weight $\theta_{avg}$.

$$\theta_{avg} = \frac{1}{N} \sum_{i=1}^{N} \theta_i \tag{2}$$

The average weights are then redistributed to each agent which then perform a soft update (Na et al., 2022) on its own previous model weights $\theta_j$.

$$\theta_{j+1} = \tau\theta_j + (1 - \tau)\theta_{avg} \tag{3}$$

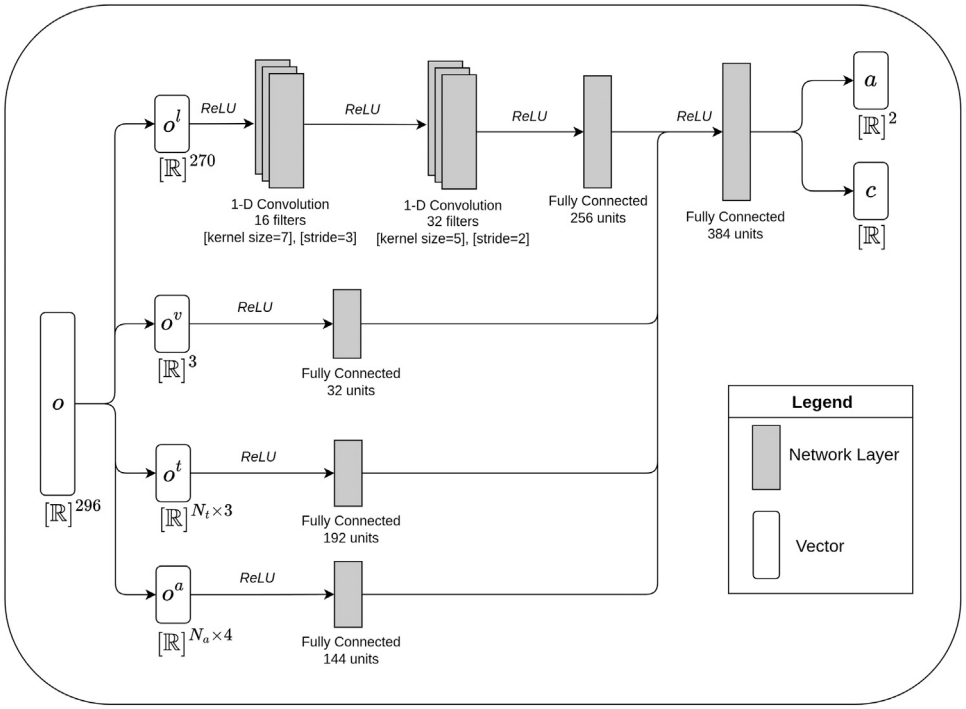



**Fig. 2.** The proposed neural network architecture for DRL agents with detailed parameters.

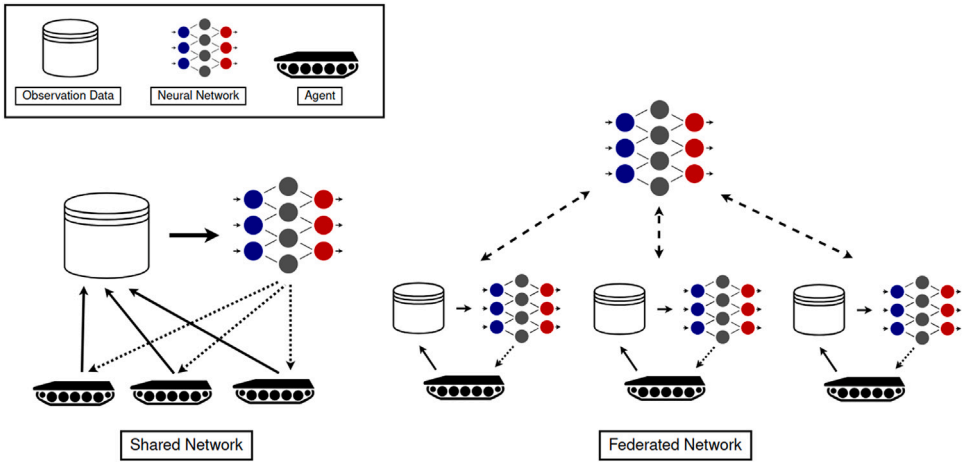



**Fig. 3.** Two multi-agent learning architectures adopted in this work.

These soft updates are scaled by $\tau$. The closer this value is to $\tau = 1$ the more influence the aggregated policy will have on each individual agent. This multi-agent training approach has also been shown in prior research to improve training stability and reduce occurrences of performance collapse (Na et al., 2022). Utilising the federated approach also requires less communication bandwidth during training due to the network weights being orders of magnitude less data than the observation vectors of each agent. However, these benefits come at the cost of a reduced convergence rate, especially if the swarm is large.

### 3.4. Simulation for training

Training is conducted in a simulation developed within a Godot environment (Godot Foundation, 2025), utilising the Jolt Physics engine (Rouwe, 2025). The UGV's are modelled as a rigidbody with two rows of idler-wheel components, each of which are individually modelled as standard second order systems (i.e. consisting of a single mass, spring, and damper). The total model was parameter fit to empirically collected data provided by Cyborg Dynamics Engineering (2025), illustrated in Fig. 4.

Integration with RL algorithms is done utilising the Godot RL Agents plugin Beeching et al. (2021) that exposes the simulation as a Gymnasium (Towers et al., 2024) environment in Python. The physics simulation is calculated in parallel to the RL agent pipeline and was run at a frequency of 100 Hz. The algorithm for stepping the simulation environment at each timestep is described by Algorithm 1.

### 3.5. Multi-agent curriculum learning towards real-world scenarios

This subsection details the first two stages in our curriculum learning, going from single agent movement to multi-agent pathfinding.

This problem environment was defined by multi-agent nonholonomic differential drive UGVs that must avoid static obstacles/terrain features to navigate to a point. The actions provided by the policy

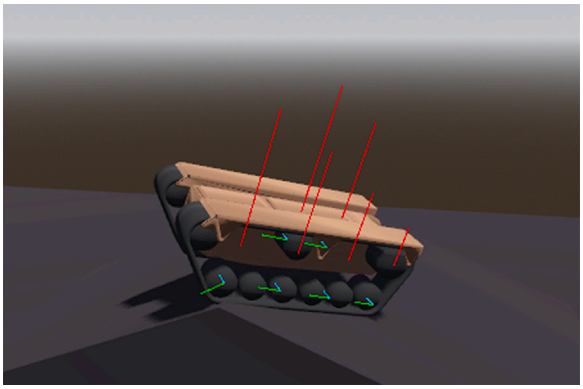


**Fig. 4.** UGV on a ramp with visible suspension (red), traction (green), and friction (blue) force lines.

**Algorithm 1** Simulation Step during Training

```
/* Compute actions for each active agent */
for all active agents i do
    A_i ← π(O_i)
end for

Step physics engine using {A_i}

/* Apply rewards and update observations */
for all active agents i do
    Apply reward r_i as per Eq. (5), Eq. (7), or Eq. (11)
    if episode terminated then
        Deactivate agent i
    else
        O'_i ← new observation
    end if
end for

/* Optimise policy based on Shared or Federated implementations */
π' ← new policy using r_i

if no agents active then
    Reset environment
end if
```

($v$ and $\omega$ for the linear and angular velocities) were mixed using a differential drive model to produce left $v_l$ and right $v_r$ track velocity setpoints as shown in Eq. (4). The values of $v$ and $\omega$ were bound by the policy to exist within the ranges of $[0.0, 1.0]$ and $[-1.0, 1.0]$ respectively.

$$\begin{aligned} v_l &= (v + 0.5\,\omega) \cdot v_{max} \\ v_r &= (v - 0.5\,\omega) \cdot v_{max} \end{aligned} \tag{4}$$

The max speed $v_{max}$ was set to 5 km/h across all agents.

### 3.5.1. Stage one: Single agent movement

To describe the learning curriculum applied to the agents, we denote $r$ to be the reward given to an agent at any given timestep, $r_x$ to be a component reward function, and $q_x$ to be a constant reward scaling parameter for a given event $x$. The first stage is a scenario with a single agent and no obstacles. The reward schema for this stage is defined in Eq. (5).

$$r = \begin{cases} q_{target} & \text{goal reached} \\ q_{to} & \text{out-of-bounds or timeout} \\ r_d & \text{otherwise} \end{cases} \tag{5}$$

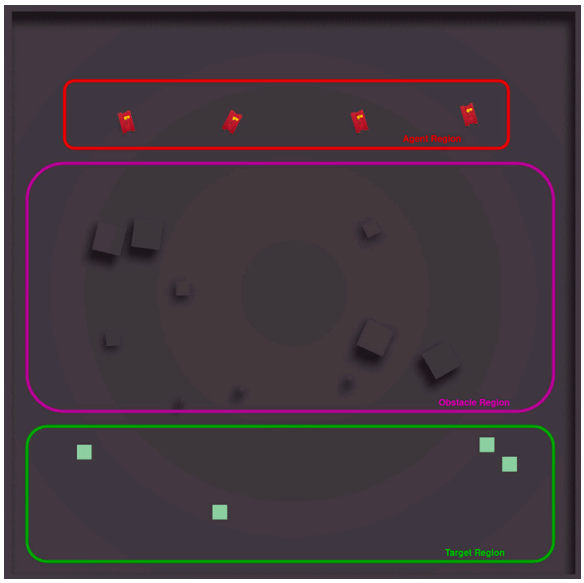



**Fig. 5.** The stage two environment showing possible obstacle and target spawn regions.

where $r_d$ is an intermediary reward function to incentivise moving towards the goal, shown in Eq. (6).

$$r_d = \begin{cases} (l^* - l) \cdot q_d & l < l^* \\ 0 & \text{otherwise} \end{cases} \tag{6}$$

where $q_{target}$ is a terminal reward received if the agent has made contact with a target, $q_{to}$ is the (negative) reward for going out of the bounds or timing out, $l$ is the current euclidean distance to the goal, and $l^*$ is the closest achieved euclidean distance to the goal, and $q_d$ is a scaling coefficient that rewards being closer to the target than the agent has been before. This reward structure is designed to incentivise consistent progress; by only rewarding the agent when it sets a new "closest distance" to the target, it prevents policies that learn to gain rewards by merely circling the target without reaching it.

### 3.5.2. Stage two: Multi-agent pathfinding

The second stage introduces obstacles and 3 other agents. These agents are constrained within a 40m×40 m field containing 10 obstacles ranging in diameter $0.3\text{ m} < d < 1.0\text{ m}$. At the beginning of each episode the obstacle and target poses are randomised with their respective possible locations shown in Fig. 5.

The starting locations of each agent are not randomised between episodes. The reward function for this environment is defined in Eq. (7).

$$r = \begin{cases} q_{goal} & \text{goal reached} \\ q_{col} & \text{collision} \\ q_{to} & \text{timeout} \\ r_l + r_m + r_d & \text{otherwise} \end{cases} \tag{7}$$

where $q_{goal}$, $q_{col}$, and $q_{to}$ are the constant hyperparameters for the goal, collision, and timeout events respectively. For non-terminal steps, the intermediary reward is defined by three functions. The goal distance reward $r_d$ is the same as defined in Eq. (6). The laser distance reward $r_l$ punishes the agent for getting too close to obstacles. It is found using an array of LiDAR scan distances $\psi$ for the current timestep and a desired minimum distance to any obstacle $l_0$, scaled by $q_l$, as shown in Eq. (8),

$$r_l = \begin{cases} (l_0 - \min \psi) \cdot q_l & \min \psi < l_0 \\ 0 & \text{otherwise} \end{cases} \tag{8}$$



**Fig. 6.** UGV extinguishing a fire in simulation.

The movement reward $r_m$ punishes movements taken throughout the episode and is found using the agent's actions at each timestep, shown in Eq. (9), with a movement reward scaling coefficient $q_m$. This incentivises the agents to complete the tasks without redundant moves.

$$r_m = (v + \omega) \cdot q_m \tag{9}$$

The logic behind this multi-part reward is twofold: the laser distance penalty ($r_l$) encourages the policy to learn a natural safety margin around obstacles rather than simply avoiding them at the last moment, while the movement penalty ($r_m$) incentivises energy and time efficiency by discouraging redundant or unnecessary actions.

### 3.6. An application in collaborative firefighting

This subsection builds on top of the previous with a third stage of the curriculum focused on a firefighting scenario. This firefighting scenario was adapted from an existing case study, Galhenage et al. (2025). The problem environments are defined by a team of nonholonomic differential drive UGVs that must avoid static obstacles/terrain to navigate and extinguish fires. Fires may require a single UGV or multiple to extinguish (see Fig. 6).

#### 3.6.1. Fire dynamics

Each UGV is assumed to be equipped with a vision pipeline and fire monitor control system that allows a UGV to automatically begin hosing a fire if the UGV's distance from a fire $d$ lies between a minimum and maximum length, $l_{min} < d < l_{max}$. Each fire's intensity $I$ begins at a starting value $I_0$ and grows continually at a fire growth rate $i_f$ upto a maximum value $I_{max}$ as described in Eq. (10). For every agent currently hosing a fire $x$, it will reduce the fire intensity by $i_h$.

$$I[k+1] = \min\Big(I[k] + i_f + (x \cdot i_h),\ I_{max}\Big) \tag{10}$$

A fire is considered *extinguished* if it is intensity drops below its starting value (i.e. $I < I_0$). Scenarios where the fire growth rate is greater than the hose rate ($i_f > i_h$) will require collaboration from multiple agents ($x > 1$) to solve.

#### 3.6.2. Stage three: Multi-agent collaborative firefighting

The training curriculum for this scenario is 3-stage, bootstrapping the policy generated from the two stages of the multi-agent pathfinding of Sections 3.5.1 and 3.5.2 into a third stage involving the fire-based targets instead of the navigation targets. The maximum number of targets for an agent to receive was chosen to be $N_t = 4$ as this aligns with the operational environment of this project, however, this number could be extended to allow the agent to receive more simultaneous data about the mission state.

**Table 2**
PPO hyperparameters.

| Optimiser | Adam |
|---|---|
| Learning Rate | 3e−5 |
| Discount $\gamma$ | 0.99 |
| Sample Period T [s] | 0.5 |
| Max Episode Time [s] | 80 |
| GAE parameter $\lambda$ | 0.95 |
| PPO clipping $\epsilon$ | 0.2 |
| PPO minibatch size | 2048 |
| Total Timesteps | 1e6 |

The reward schema for this stage is defined in Eq. (11).

$$r = \begin{cases} q_{col} & \text{collision} \\ q_{to} & \text{timeout} \\ r_l + r_m + r_h & \text{otherwise} \end{cases} \tag{11}$$

where $r_h$ is a intermediary reward to encourage hosing actions, as shown in Eq. (12). If the agent is currently hosing a fire, the $q_h$ reward will be applied. If a fire is extinguished, all agents currently hosing that fire will receive a flat reward of $q_{ext}$ instead of $q_h$. If the agent is not currently hosing, no hosing reward is applied. An additional group reward for extinguishing all fires was considered, however, the use of a reward function that focused on individual performance instead was found to produce acceptable collaborative behaviour from the agents, negating the need for group rewards and analysis into credit assignment. This agent-centric design was a deliberate choice to simplify the credit assignment problem, a significant challenge in MARL, by ensuring each agent is rewarded for its direct, measurable contribution to the mission.

$$r_h = \begin{cases} q_{ext} & \text{fire extinguished} \\ q_h & \text{is hosing} \\ 0 & \text{else} \end{cases} \tag{12}$$

The training environment mimics the previous section, retaining the randomised obstacles but replacing the target navigation goals with fires.

## 4. Experimental results

The PPO training hyperparameters are shown in Table 2. The max episode time was chosen based on a reasonable time estimate for a human operator to complete the tasks reliably. The GAE parameter $\lambda$ balances between bias and variance when estimating advantage, while the PPO clipping $\epsilon$ and minibatch size affect how much the policy can change during each update and the amount of training data fed into the optimisation stage at a time when calculating gradients, respectively. The values for these parameters were chosen based on the existing evaluations in Schulman et al. (2017).

### 4.1. Stage one results

Stage one used the reward parameters shown in Table 3; however, the training stability appeared to be similar using a wide range of reward scaling values, so adjusting the values below has little effect on training and testing results. Furthermore, across all of the stages of the learning curriculum, the timeout reward $q_{to}$ was found to have little effect and so was set to 0.00.

Stage one approached a reasonable success rate within 5000 episodes of training, shown in Fig. 7, validating the methodology for simple navigational tasks.

After stage one training was complete, the policy had achieved a 100% success rate in an evaluation consisting of 100 episodes. This initial result served as a crucial baseline, confirming that the core network architecture and learning algorithm were effective for fundamental point-to-point navigation.

**Table 3**
Stage one reward parameters.

| Parameter | Description | Value |
|---|---|---|
| $q_{target}$ | target reached | 1.00 |
| $q_{to}$ | timed out | 0.00 |
| $q_d$ | progress to goal | 0.04 |

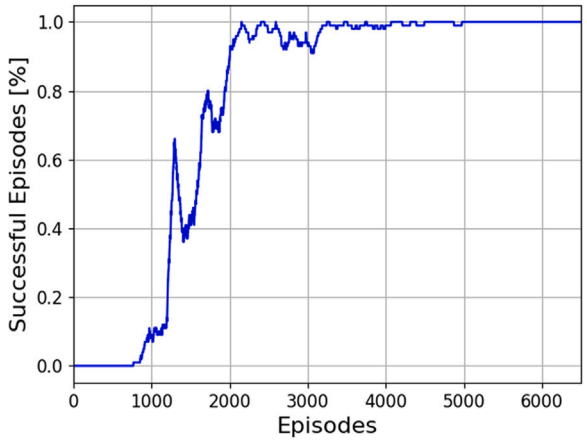



**Fig. 7.** Stage one success rate during training.

**Table 4**
Stage two reward parameters.

| Variable | Meaning | Value |
|---|---|---|
| $q_{goal}$ | goal reached | 1.00 |
| $q_{col}$ | collision | −1.00 |
| $q_{to}$ | timed out | 0.00 |
| $q_l$ | proximity to obstacles | −0.01 |
| $q_m$ | movement | 0.00 |
| $l_0$ | acceptable distance from obstacles [m] | 2.50 |
| $q_d$ | progress to goal | 0.04 |

### 4.2. Stage two results

The trained policy from stage one was bootstrapped to the stage two scenario. The reward parameters used during stage two based on parameter tuning in training are shown in Table 4. The movement $q_m$ reward was found to not make a noticeable difference in the performance of the policy within this stage of the curriculum so was left as 0.00.

The training was conducted utilising both shared and federated network training approaches. To record a more representative baseline for the performance of each method, the shared and federated training approaches each had 12 training instances run utilising different training seeds for optimisation and environment randomisation, resulting in 24 different policies. The training curves for 3 shared and 3 federated network policies are shown in Fig. 8.

When experimenting with different reward parameters, it was found that the federated approach had a greater region of convergence, whereby it was able to achieve a positive result with a wider range of reward parameters than the shared network. Furthermore, the range of rewards during training under the federated approach tended to be smaller across different seeds (as seen in Fig. 8), indicating the federated approach may have higher training stability. This is a similar result to that shown in prior work done by Na et al. (2022).

To evaluate these policies, a "successful" run was defined to be one where the agents reached their respective targets without any collisions with any other agents or obstacles. The 12 policies created using each training approach were evaluated over 100 episodes each, yielding 1200 test runs for each approach. The shared network policies in aggregate consistently had a higher success rate in evaluation than the federated policies, achieving a mean success rate of 95.83% ($\sigma = 0.52$) on average compared to the federated policies which had a mean of 90.00% ($\sigma = 3.31$). This corroborated the results of Na et al. (2022) with the federated approach having a more consistent and stable training curve, however, in this case these benefits had occurred at the cost of converging at a significantly slower rate. Failures across all policies most often occurred due to agent-agent collisions as opposed to agent-obstacle collisions. This failure mode suggests that while the LiDAR-based policy for avoiding static obstacles is highly robust, the more complex, decentralised task of negotiating passage with other moving agents in confined spaces remains the primary challenge. It highlights that inter-agent coordination, rather than simple obstacle avoidance, is the harder problem to solve.

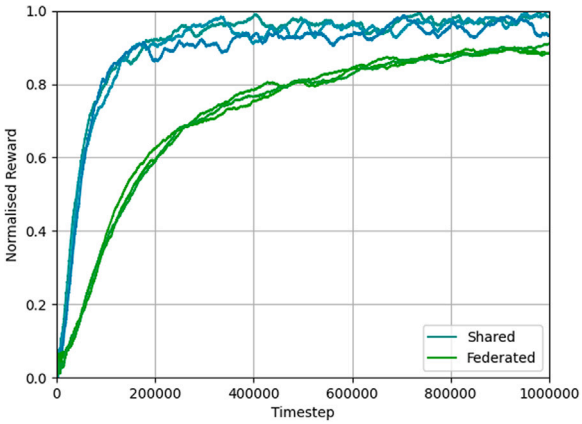



**Fig. 8.** Stage two shared and federated learning comparison in training.

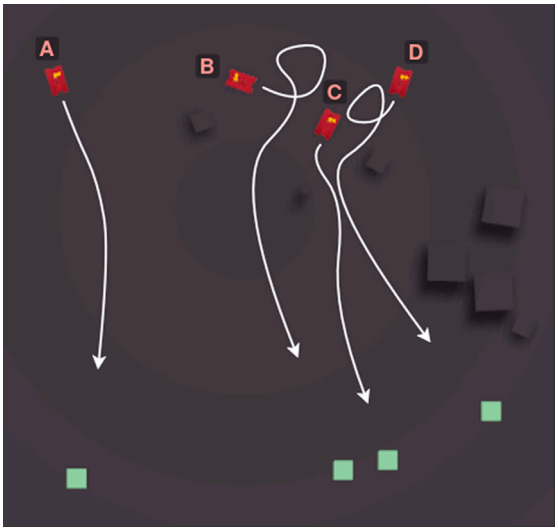
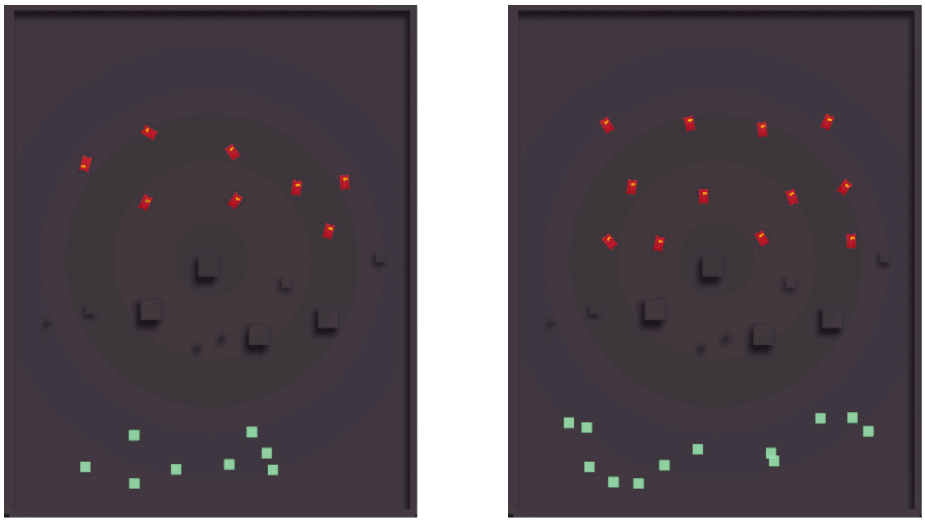



**Fig. 9.** Stage two example agent trajectory, illustrating emergent yielding behaviours during navigation.

The method was able to recreate the emergent yielding rules as demonstrated previously by Surmann et al. (2020) whereby when two agents approach a narrow passage between obstacles, one or both agents will back away and search for another entrance. Fig. 9 shows an example trajectory where three agents (B, C, and D) approach the same narrow passage between obstacles. Agents B and D circle around to find another approach while agent C immediately traverses the passage.

An ablation analysis was conducted where the two intermediary reward components were removed and training was rerun, with each policy's evaluation performance shown in Table 5. In particular, these



(a) 8 agents. (b) 12 agents.

**Fig. 10.** Navigation scenario with additional agents.

**Table 5**
Stage two ablation results on success rate of 1200 testing runs in randomised environment, comparing with results from prior work in static and randomised environments.

| | Success (%) | STD | CI (95%) |
|---|---|---|---|
| Our method w/ shared network | 95.83 | 0.52 | 95.242–96.418 |
| Our method w/ Federated learning | 90.00 | 3.31 | 86.254–93.746 |
| No randomised obstacles in training | 91.50 | 2.41 | 88.773–94.227 |
| $r_l(kT)$ removed from training | 91.17 | 1.94 | 88.975–93.365 |
| $r_d(kT)$ removed from training | 95.33 | 1.77 | 93.327–97.333 |
| | | | Success (%) |
| Jestel et al. (2021) static 'hall' (16 agents) | | | 98.38 |
| Jestel et al. (2021) static 'robohall' (14 agents) | | | 98.28 |
| Jestel et al. (2021) randomised 'hall' (4 agents) | | | 71.00 |
| Jestel et al. (2021) randomised 'robohall' (4 agents) | | | 51.85 |

were the laser distance reward $r_l$ responsible for minimising distance to obstacles, and the shorter distance reward $r_d$ incentivising the agent to approach the target. The randomisation of the obstacle locations/orientations was also ablated (i.e. obstacles did not move between training episodes, but still were randomised during evaluation) and shown in Table 5.

These results can be compared with those reported by Jestel et al. (2021) in their "hall" and "robolab" scenarios. While their approach achieved success rates exceeding 98% in known environments, performance declined to between 50% and 70% when evaluated in previously unseen settings — conditions that are comparable to the randomised objects used in our environment. By contrast, our method achieved a mean success rate of 95.83% (4 agents), although, specific differences in the environments may impact this figure.

Further testing on the stage two policy was performed by utilising the same evaluation scenario but with additional agents in the team and an extended starting area, as shown in Fig. 10.

The performance of the increased swarm sizes are shown in Table 6. From these results, we can see that the occurrences of collisions increases as you increase the number of agents needing to pass through the narrow passages within the time limit. It is worth noting for implementation that these results are dependent on ensuring the randomisation of obstacles do not 'line up' to form large impassable walls and that targets are not too clustered together (both may make it impossible for the certain agents to achieve the goal). Note that 24 agents is much larger than any typical team of BIA5 (2021) UGVs in prior real-world scenarios.

**Table 6**
Stage two scalability results.

| Agents | Targets | Success (%) | STD | CI (95%) |
|---|---|---|---|---|
| 4 | 4 | 95.83 | 0.52 | 95.242–96.418 |
| 8 | 8 | 84.50 | 4.33 | 79.600–89.400 |
| 12 | 12 | 81.11 | 4.41 | 76.120–86.100 |
| 24 | 24 | 69.36 | 3.64 | 65.241–73.479 |

When adding more agents into the scenario of the same environment size, the number of collisions increases as the area becomes crowded, reducing the success rate. This indicates that the yielding rules developed may not fully consider scenarios with large amounts of agents all traversing through the same narrow hallway with the current training parameters. Including such scenarios with larger swarm tasks into the training curriculum may improve these results further.

The Stage Two results successfully demonstrated a robust multi-agent navigation policy. The shared-network architecture was shown to be superior to the federated approach, achieving a 95.83% collision-free success rate in dynamic environments, outperforming comparable prior work. The ablation study confirmed that all components of the reward function were essential for this performance. The primary limitations identified were a degradation in performance in high-density scenarios and a tendency for failures to be caused by agent-agent collisions, highlighting inter-agent coordination as a key challenge.

### 4.3. Stage three results

The control shared-network policy with four agents from stage two was bootstrapped to the stage three scenario. The reward parameters used during stage three are shown in Table 7. These parameters were tuned during training to obtain the optimal training performance. It was found that, unlike in stage 2, the movement reward $q_m$ was found to improve performance and so was changed for this stage of training.

The training was conducted utilising both shared and federated network training approaches. The shared and federated training approaches each began with the stage two policy weights loaded and had 12 instances run utilising different training seeds for optimisation and environment randomisation, resulting in 24 different policies for stage three evaluation. The training curves for 3 shared and 3 federated network policies are shown in Fig. 11.

**Table 7**
Stage three reward parameters.

| Variable | Meaning | Value |
|---|---|---|
| $q_{col}$ | collision | −1.00 |
| $q_{to}$ | timed out | 0.00 |
| $q_l$ | proximity to obstacles | −0.01 |
| $l_0$ | acceptable distance from obstacles [m] | 2.50 |
| $q_m$ | movement | −0.005 |
| $q_{ext}$ | fire extinguished | 1.0 |
| $q_h$ | fire hosing | 0.005 |

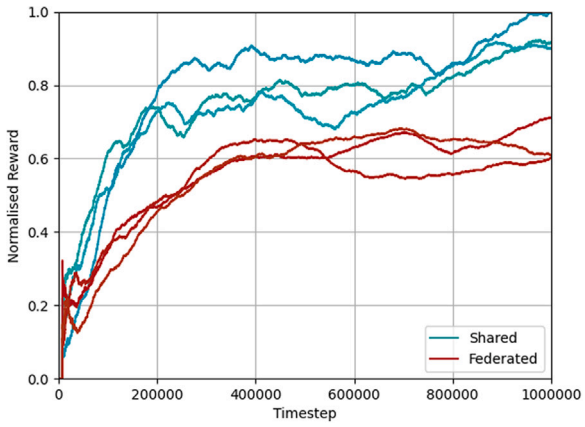



**Fig. 11.** Stage three smoothed reward during training.

Training performance during stage three was shown to contain far greater variation when compared to stage two.

The 12 policies for the stage two shared network and stage three shared network approaches were each evaluated over 100 episodes in the training scenario with 2 fires. The stage three training resulted in a policy with a mean mission success rate of 99.67% ($\sigma = 0.58$) which was a large improvement from the stage two policy which when applied directly to the stage three scenario evaluation only achieved a mean success rate of 77.92% ($\sigma = 12.73$) with much higher variance in results. This large variation in success of the stage two policy can be attributed to the stochastic arrangement of obstacles. The team's ability to extinguish the fires depended on whether the obstacles happened to facilitate the agents splitting into efficient formations to target different fires, which only occurred in some episodes. The major observed behaviour improvements for the stage three policy occurred within the higher-level task allocation, whereby the agents would split into pairs that targeted separate fires (as illustrated in Fig. 12), as this produced a far higher likelihood of extinguishing all the fires within the set time limit by taking advantage of the number of agents within the team to minimise the time spent travelling between fires (see Table 8).

The two policies were also evaluated in scenarios of increasing difficulty, whereby additional fires were added and the time constraints kept the same for the four agents. These aggregated success results are shown in Table 9.

As the difficulty of the scenario was increased by the inclusion of additional fires to extinguish under the same time constraint, the stage three policy demonstrated smaller decreases in performance compared to the stage two policy, while also maintaining a more reasonable standard deviation of $\sigma = 5.63$ at the lowest end in the four fire scenarios. Furthermore, a "dropout" modification was tested within the 4 fire scenario whereby the nearby agent observations $o_a$ were disconnected during the scenario. Although the scenario was still able to be completed within most tests at a success rate of 81.67%, the removal of the ability of the agent to consider the relative positions of the other agents when deciding which fire to target had caused a notable decrease in performance.

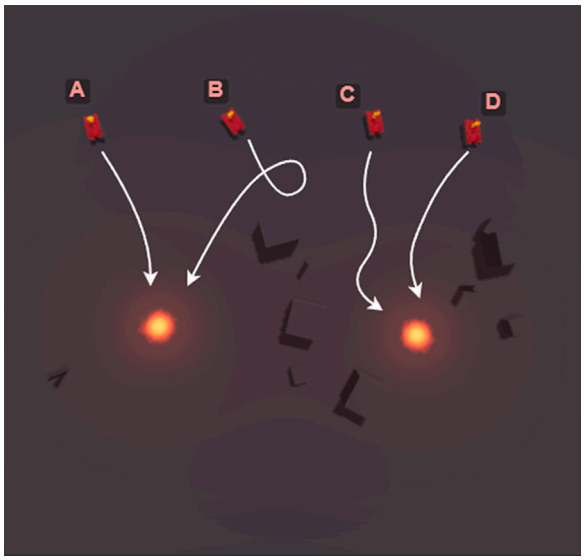



**Fig. 12.** Stage three example agent trajectory, illustrating higher-level task-allocation behaviours concurrently with existing navigation behaviours.

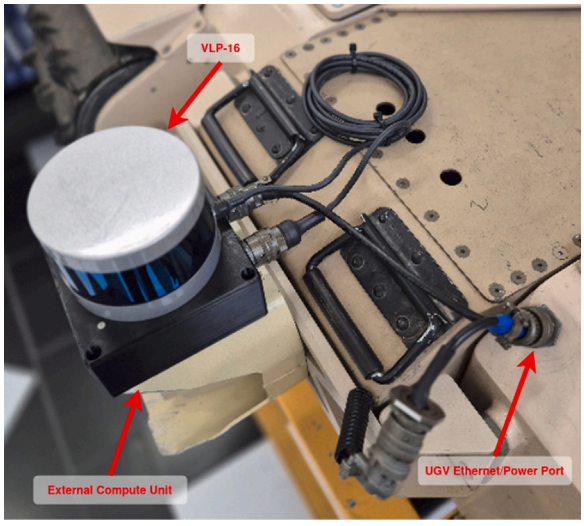
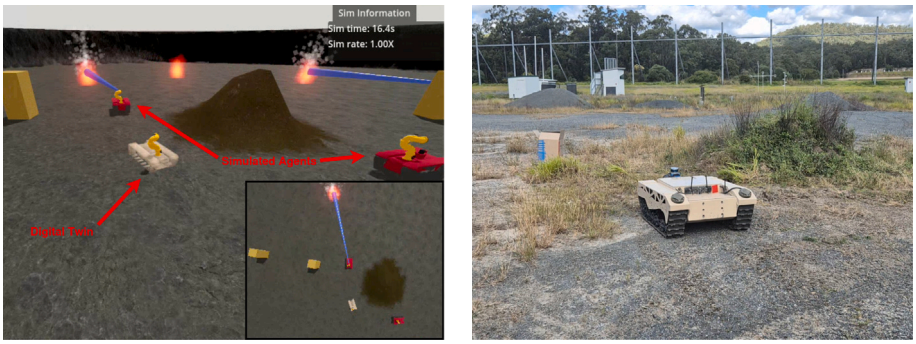



**Fig. 13.** External hardware module.

**Table 8**
Stage three success rate for different scenarios with four agents in 1200 test runs. "Dropout" simulates a scenario where agent communication is disabled..

| Scenarios | Mean (%) | STD | CI (95%) |
|---|---|---|---|
| Stage Two Policy | | | |
| 2 Fires | 77.92 | 12.73 | 63.515–92.325 |
| 3 Fires | 63.58 | 14.00 | 47.738–79.422 |
| 4 Fires | 55.25 | 12.03 | 41.637–68.863 |
| Stage Three Policy | | | |
| 2 Fires | 99.67 | 0.58 | 99.025–100.32 |
| 3 Fires | 90.42 | 2.77 | 87.286–93.554 |
| 4 Fires | 86.75 | 5.63 | 80.379–93.121 |
| 4 Fires (Dropout) | 81.67 | 2.52 | 80.972–82.368 |

The results from Stage Three provide strong validation for our multi-stage curriculum learning approach. The specialised fine-tuning resulted in a dramatic improvement in mission success, increasing the rate from 77.92% to 99.67% in the two-fire scenario. This was attributed to the policy learning high-level collaborative tactics, such




(a) Simulation (b) Physical

**Fig. 14.** Hybrid demonstration.

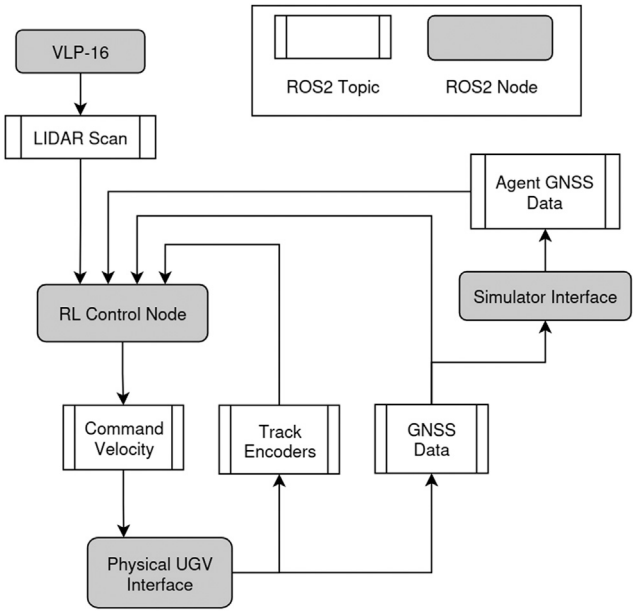



**Fig. 15.** ROS2 node and topic architecture for hybrid evaluation.

as dividing the team to address multiple targets in parallel. Furthermore, the final policy demonstrated significantly more robust performance as the scenario difficulty increased, confirming the value of the final curriculum stage for developing sophisticated, mission-specific behaviours.

### 4.4. Sim-to-real transfer

The RL policy was transferred to a physical UGV. The chosen robotic platforms were the '1050' UGV platforms provided by Cyborg Dynamics Engineering (2025). These platforms have been proven in operational environments where they are used to extinguish fires through teleoperation (BIA5, 2021). Due to the high cost of the UGV, we could only experiment with one machine. However, the successful transfer to one physical machine and demonstration that it interacts with the other simulated agents as expected show that the proposed multi-agent method should also scale to multiple physical machines. The robotic platform is capable of transmitting GNSS and wheel encoder data over ROS2 (Macenski et al., 2022) and receives `$geometry_msgs/Twist$` control commands as a linear and forward velocity. The policies were realised as a ROS2 node on the vehicle ecosystem. An external hardware module – shown in Fig. 13 – was developed that included a compute unit to run the RL policy as a ROS2 node and a VLP-16 LiDAR module. The 3D pointcloud data from the LiDAR module was sandwiched into a 270 degree 2D laser scan, with the algorithm taking the minimum laser distance of each vertical segment of points. This hardware module was mechanically integrated to the platform and connected via an external UGV port providing an Ethernet interface to the local vehicle network.

**Table 9**
Hybrid evaluation results.

| Metric ($n = 10$) | Occurrences |
|---|---|
| All fires doused | 10 |
| No Physical Collisions | 7 |
| No Simulation Collisions | 10 |

The policy's navigation and multi-agent fire-fighting capability was then validated in a hybrid physical-simulated environment. As only one physical UGV was available due to budget constraints, three other agents were simulated in a hybrid physical–cyber evaluation as shown in Fig. 14. The ROS2 data architecture is described in Fig. 15, where the physical UGV's geospatial location is transmitted to a simulation where it appears as a digital twin for the three other simulated agents to avoid during navigation and include as a resource during task allocation for fire fighting scenarios.

Through this architecture the transference of the RL policy from simulation to physical was demonstrated. During the hybrid evaluation, the dispersion behaviour from the simulation was frequently recreated, with two of the simulation agents dispersing from the physical agent and one of the simulated agents following the physical agent.

The physical evaluation was done in an environment with only three obstacles, two cardboard boxes and a single large dirt/grass mound, as shown in Fig. 14b. A total of 10 hybrid evaluations were performed.

Due to the simpler navigation task (fewer obstacles) the three simulated agents were able to easily avoid all the obstacles and other agents with no collisions.

The physical agent had collisions with obstacles in three instances while traversing to fires. During these collisions, a limitation of the LiDAR pipeline was found whereby long grass close to the vehicle was sandwiched into the presence of an false positives as well as slight movements in the obstacles from the wind causing false negatives. This erroneous data was then fed to the RL pipeline causing issues. When moving obstacles away from the long grass and fixing the obstacles these collisions were avoided, however, demonstrate a potential area for further investigation. Future work on this project could investigate the use of more sophisticated pointcloud LiDAR processing algorithms to provide a more robust performance.

Due to the precise control loop of the UGV hardware (operating on the order of ms) and the large sample period of RL actions (running at 2 Hz), no latency issues were encountered during the sim to real transfer.

Furthermore, due to the size of the UGV and obstacles being orders of magnitude larger than the accuracy of the sensors, issues regarding sensor noise were not encountered during the sim to real transfer.

This section demonstrated a successful proof-of-concept for the sim-to-real transfer of the learned multi-agent policy. The policy was successfully deployed on a (BIA5, 2021) '1050' UGV platform using a ROS2-based architecture. The proposed hybrid physical–cyber evaluation method was validated, and crucially, key emergent behaviours learned entirely in simulation were observed to be successfully recreated by the physical UGV interacting with its simulated teammates. This provides strong evidence for the viability of the training and validation pipeline, even when limited to a single physical hardware unit.

## 5. Conclusion

In this paper, we demonstrated that a multi-agent DRL policy, trained through a structured three-stage curriculum, can learn to perform both low-level navigation and high-level collaborative tasks. Through evaluations in simulation and in a hybrid physical-cyber environment, we analysed key reward shaping characteristics and showed that our approach produces a unified policy capable of emergent, decentralised coordination. We successfully validated the sim-to-real transfer of this policy onto a physical UGV, confirming the operational potential of our approach for practical case studies like autonomous navigation and firefighting.

The successful sim-to-real transfer and robust performance in collaborative tasks confirm the operational potential of our approach. For emergency response applications, this is a significant step towards deploying autonomous swarms that can act as a cohesive force multiplier. The ability of the agents to self-organise and allocate tasks – such as splitting into teams to extinguish multiple fires – reduces the cognitive load on human operators and allows for hazards to be addressed in parallel and autonomously, enhancing the safety and efficiency of response efforts in dangerous environments.

For future research, a comparison of the combined high-low level decision-making approach against a more traditional low-level stack with high-level decisions optimised by a DRL policy would provide further insights into the most effective uses for these policies. The ideas presented in this article could also be expanded upon through further insight into improving the sim-to-real transfer learning scenarios whereby more empirical noise/disturbances from the physical scenarios are identified and implemented into the training curricula. Furthermore, an investigation into adding additional layers to the neural network architecture could potentially yield greater multi-agent collaborative performance in these scenarios.

## CRediT authorship contribution statement

**Timothy Mead:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Zhe Wang:** Writing – review & editing, Supervision, Funding acquisition, Conceptualization. **Ernest Foo:** Writing – review & editing, Methodology, Funding acquisition, Conceptualization. **Jin Song Dong:** Writing – review & editing, Funding acquisition, Conceptualization. **Naipeng Dong:** Writing – review & editing, Methodology, Funding acquisition, Conceptualization. **Ryan Ko:** Writing – review & editing, Supervision, Funding acquisition. **Abigail M.Y. Koay:** Writing – review & editing, Methodology, Funding acquisition, Conceptualization. **Kien Nguyen:** Writing – review & editing, Methodology, Funding acquisition, Conceptualization. **Yue Xu:** Writing – review & editing, Methodology, Funding acquisition, Conceptualization. **Junae Kim:** Writing – review & editing, Writing – original draft, Methodology, Investigation, Funding acquisition, Conceptualization. **Stephen Bornstein:** Writing – review & editing, Supervision, Funding acquisition. **Zhe Hou:** Writing – review & editing, Writing – original draft, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Conceptualization.

## Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Timothy Mead, Zhe Wang, Ernest Foo, Jin Song Dong, Naipeng Dong , Ryan Ko, Abigail MY Koay, Kien Nguyen, Yue Xu, Junae Kim, Stephen Bornstein, Zhe Hou reports financial support was provided by Queensland Defence Science Alliance (QDSA). If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgment

This research is funded by the Queensland Defence Science Alliance under the project QDSA_2023_OUT/063.

## Data availability

Data will be made available on request.

## References

Alagha, A., Singh, S., Mizouni, R., Bentahar, J., Otrok, H., 2022. Target localization using multi-agent deep reinforcement learning with proximal policy optimization. Future Gener. Comput. Syst. 136, 342–357.

Alonso, E., Peter, M., Goumard, D., Romoff, J., 2020. Deep reinforcement learning for navigation in AAA video games. URL https://arxiv.org/abs/2011.04764. arXiv:2011.04764.

Baker, B., Kanitscheider, I., Markov, T., Wu, Y., Powell, G., McGrew, B., Mordatch, I., 2020. Emergent tool use from multi-agent autocurricula. URL https://arxiv.org/abs/1909.07528. arXiv:1909.07528.

Beeching, E., Dibangoye, J., Simonin, O., Wolf, C., 2021. Godot reinforcement learning agents. arXiv Preprint arXiv:2112.03636.

BIA5, 2021. BIA5's latest ATR platform integration. https://bia5.com/bia5s-latest-atr-platform-integration/.

Cyborg Dynamics Engineering, 2025. Warfighter UGV. https://www.cyborgdynamicseng.com/ugvs.

Everett, M., Chen, Y.F., How, J.P., 2021. Collision avoidance in pedestrian-rich environments with deep reinforcement learning. IEEE Access 9, 10357–10377. http://dx.doi.org/10.1109/access.2021.3050338.

Galhenage, E., Uzunov, A., Taylor, L., 2025. A bushfire scenario for hybrid multi-agent reinforcement learning research. Defence Science and Technology Group, Technical Note, DSTG-TN-2539.

Gao, L., Zhuang, Z., Tao, H., Chen, Y., Paszke, W., Stojanovic, V., 2025. A decentralized optimal iterative learning control approach with efficient computation for collaborative tracking. Internat. J. Robust Nonlinear Control http://dx.doi.org/10.1002/rnc.70020, URL https://onlinelibrary.wiley.com/doi/abs/10.1002/rnc.70020.

Godot Foundation, 2025. Godot engine. https://github.com/godotengine/godot.

Huang, S., Dossa, R.F.J., Ye, C., Braga, J., Chakraborty, D., Mehta, K., Araújo, J.G., 2022. CleanRL: High-quality single-file implementations of deep reinforcement learning algorithms. J. Mach. Learn. Res. 23 (274), 1–18, URL http://jmlr.org/papers/v23/21-1342.html.

Jestel, C., Surmann, H., Stenzel, J., Urbann, O., Brehler, M., 2021. Obtaining robust control and navigation policies for multi-robot navigation via deep reinforcement learning. In: 2021 7th International Conference on Automation, Robotics and Applications. ICARA, IEEE, pp. 48–54. http://dx.doi.org/10.1109/icara51699.2021.9376457.

Kaufmann, E., Bauersfeld, L., Loquercio, A., Müller, M., Koltun, V., Scaramuzza, D., 2023. Champion-level drone racing using deep reinforcement learning. Nat. 620.

Li, S., Wu, Y., Cui, X., Dong, H., Fang, F., Russell, S., 2019. Robust multi-agent reinforcement learning via minimax deep deterministic policy gradient. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 33, (01), pp. 4213–4220.

Li, J., Yang, Y., He, Z., Wu, H., Shi, H., Chen, W., 2024. Cournot policy model: Rethinking centralized training in multi-agent reinforcement learning. Inform. Sci. 677, 120983. http://dx.doi.org/10.1016/j.ins.2024.120983.

Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., Wierstra, D., 2019. Continuous control with deep reinforcement learning. URL https://arxiv.org/abs/1509.02971. arXiv:1509.02971.

Macenski, S., Foote, T., Gerkey, B., Lalancette, C., Woodall, W., 2022. Robot operating system 2: Design, architecture, and uses in the wild. Sci. Robot. 7 (66), eabm6074. http://dx.doi.org/10.1126/scirobotics.abm6074, URL https://www.science.org/doi/abs/10.1126/scirobotics.abm6074.

Mamond, A.W., Kundroo, M., Yoo, S.-e., Kim, S., Kim, T., 2025. FLDQN: Cooperative multi-agent federated reinforcement learning for solving travel time minimization problems in dynamic environments using SUMO simulation. Sensors 25 (3), http://dx.doi.org/10.3390/s25030911, URL https://www.mdpi.com/1424-8220/25/3/911.

Miller, N., Goulet, N., Ayalew, B., 2024. Energy-aware collaborative teaming and route planning for unmanned ground vehicles. IFAC-PapersOnLine 58 (28), 1007–1012.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., Hassabis, D., 2015. Human-level control through deep reinforcement learning. Nature 518 (7540), 529–533. http://dx.doi.org/10.1038/nature14236.

Na, S., Rouček, T., Ulrich, J., Pikman, J., Krajník, T., Lennox, B., Arvin, F., 2022. Federated reinforcement learning for collective navigation of robotic swarms. URL https://arxiv.org/abs/2202.01141. arXiv:2202.01141.

Orr, J., Dutta, A., 2023. Multi-agent deep reinforcement learning for multi-robot applications: A survey. Sensors 23 (7), http://dx.doi.org/10.3390/s23073625, URL https://www.mdpi.com/1424-8220/23/7/3625.

Peng, Z., Song, X., Song, S., Tejado, I., Stojanovic, V., 2025. Resilient consensus tracking control and vibration suppression of moving multiple helicopter flexible slung-load systems. J. Vib. Control 10775463251321551. http://dx.doi.org/10.1177/10775463251321551.

Phan, T., 2023. Emergence and Resilience in Multi-Agent Reinforcement Learning (Ph.D. thesis). LMU Munich.

Rouwe, J., 2025. Jolt physics. https://github.com/jrouwe/JoltPhysics.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal policy optimization algorithms. URL https://arxiv.org/abs/1707.06347. arXiv:1707.06347.

Shida, Y., Jimbo, T., Odashima, T., Matsubara, T., 2025. Reinforcement learning of multi-robot task allocation for multi-object transportation with infeasible tasks. In: 2025 IEEE/SICE International Symposium on System Integration. SII, IEEE, pp. 1548–1555. http://dx.doi.org/10.1109/sii59315.2025.10870902.

Su, M., Pu, R., Wang, Y., Yu, M., 2024. A collaborative siege method of multiple unmanned vehicles based on reinforcement learning. Intell. Robot. 4 (1), http://dx.doi.org/10.20517/ir.2024.03, URL https://www.oaepublish.com/articles/ir.2024.03.

Surmann, H., Jestel, C., Marchel, R., Musberg, F., Elhadj, H., Ardani, M., 2020. Deep reinforcement learning for real autonomous mobile robot navigation in indoor environments. URL https://arxiv.org/abs/2005.13857. arXiv:2005.13857.

Tan, D.M.S., Ma, Y., Liang, J., Chng, Y.C., Cao, Y., Sartoretti, G., 2025. IR2: Implicit rendezvous for robotic exploration teams under sparse intermittent connectivity. URL https://arxiv.org/abs/2409.04730. arXiv:2409.04730.

Towers, M., Kwiatkowski, A., Terry, J., Balis, J.U., Cola, G.D., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., Perez-Vicente, R., Pierré, A., Schulhoff, S., Tai, J.J., Tan, H., Younis, O.G., 2024. Gymnasium: A standard interface for reinforcement learning environments. URL https://arxiv.org/abs/2407.17032. arXiv:2407.17032.

Upadhyay, A., Abafat, S., Baradaranshokouhi, Y., Lu, X., Jing, Y., Li, J., 2024. Federated reinforcement learning enhanced human-robotic systems: A comprehensive review. In: 2024 IEEE International Conference on E-Business Engineering. ICEBE, pp. 145–151. http://dx.doi.org/10.1109/ICEBE62490.2024.00031.

Wang, C., Wang, J., Wei, C., Zhu, Y., Yin, D., Li, J., 2023. Vision-based deep reinforcement learning of UAV-ugv collaborative landing policy using automatic curriculum. Drones 7 (11), http://dx.doi.org/10.3390/drones7110676, URL https://www.mdpi.com/2504-446X/7/11/676.

Wang, Z., Zhao, S., 2025. Sim-to-real transfer in reinforcement learning for maneuver control of a variable-pitch MAV. IEEE Trans. Ind. Electron. 1–10. http://dx.doi.org/10.1109/TIE.2025.3558030.

Wei, C., Li, Y., Ouyang, Y., Ji, Z., 2023. Deep reinforcement learning with heuristic corrections for ugv navigation. J. Intell. Robot. Syst. 109 (1), http://dx.doi.org/10.1007/s10846-023-01950-y.

Wen, S., Wen, Z., Zhang, D., Zhang, H., Wang, T., 2021. A multi-robot path-planning algorithm for autonomous navigation using meta-reinforcement learning based on transfer learning. Appl. Soft Comput. 110, 107605. http://dx.doi.org/10.1016/j.asoc.2021.107605, URL https://www.sciencedirect.com/science/article/pii/S1568494621005263.

Xiao, C., Lu, P., He, Q., 2021. Flying through a narrow gap using end-to-end deep reinforcement learning augmented with curriculum learning and Sim2Real. URL https://arxiv.org/abs/2108.12869. arXiv:2108.12869.

Yu, C., Yang, X., Gao, J., Chen, J., Li, Y., Liu, J., Xiang, Y., Huang, R., Yang, H., Wu, Y., Wang, Y., 2023. Asynchronous multi-agent reinforcement learning for efficient real-time multi-robot cooperative exploration. In: Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems. AAMAS '23, International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, pp. 1107–1115.

Zhang, K., Niroui, F., Ficocelli, M., Nejat, G., 2018. Robot navigation of environments with unknown rough terrain using deep reinforcement learning. In: 2018 IEEE International Symposium on Safety, Security, and Rescue Robotics. SSRR, pp. 1–7. http://dx.doi.org/10.1109/SSRR.2018.8468643.